

Task 4

After a tedious ros2 introduction, let's move to simulation phase which will make us to test our bot before hardware.

1. Task 4-a: Robot Modeling using URDF

The goal of this task is to use ROS to:

- construct a basic robot model using the Unified Robot Description Format (URDF),
- define physical links and joints,
- and assign proper collision, visual, and inertial properties.

Problem Statement

The robotics division of Cyborg is moving the physical line-following robot designed during the induction process into a simulated environment. Before the robot can be driven in a simulator, its physical structure must be accurately modeled.

Your task is to design the urdf of a line following bot (similar to the one you designed during T-o-q).

Robot Description

The URDF model must accurately reflect the induction line-following bot and contain:

- a main chassis (**base_link**),
- a left drive wheel,
- a right drive wheel,
- and required caster wheel for balance.

Task Requirements

Your program must:

1. Create a URDF file defining all the required links and joints.
2. Connect the drive wheels to the chassis using **continuous** joints.
3. Connect the caster wheel(s) using **fixed** or **continuous** joints depending on your design approach.
4. Define proper **<visual>** tags for appearance.
5. Define accurate **<collision>** tags to ensure the robot interacts physically with the environment.
6. Define **<inertial>** tags with realistic mass and inertia matrices for every link.
7. Visualize it in Rviz and Gazebo.

Expected Output Format (Task 4-a)

For this specific sub-task, your output is the completed `.urdf` file. You should be able to parse this file without any syntax errors using ROS URDF parsing tools.

Implementation and Evaluation

Write your URDF logic and comment it at the bottom of your URDF in few lines. **Your logic description should be written by you only, no AI allowed here.**

IMPORTANT: Do NOT ignore the collision and inertia matrices. You can test your initial URDF visually by launching it in RViz before moving on to Gazebo.

2. Task 4-b: Gazebo Simulation and Teleop Control

The goal of this task is to use ROS and Gazebo to:

- integrate ROS simulation plugins,
- spawn a URDF model in a physics environment,
- and simulate robot kinematics using manual teleop control.

Problem Statement

Now that the robot's physical blueprint (URDF) is complete, it must be deployed into a physics simulator to test its mobility. The robot requires a differential drive controller to translate wheel rotations into actual movement across the environment.

Your task is to modify your existing URDF by adding a differential drive plugin, spawn the robot in Gazebo, and successfully navigate it using keyboard commands.

Task Requirements

Your implementation must:

1. Take the URDF generated in Task 4-a.
2. Add the Gazebo differential drive plugin to the model.
3. Correctly map the left and right wheel joint names within the plugin's configuration parameters.
4. Set the correct wheel separation and wheel diameter in the plugin.
5. Launch the robot in an empty Gazebo world.
6. Make sure to run the ros-gazebo bridge before running the teleop key otherwise no `cmd_vel` topic will be published.

When you launch teleop key it publishes ros topic `cmd_vel`, but it needs to be translated into gazebo topic form of `cmd_vel`. That's why the bridge is ran in the terminal. You can search more regarding it :)

None

```
ros2 run ros_gz_bridge parameter_bridge  
/cmd_vel@geometry_msgs/msg/Twist@gz.msgs.Twist
```

7. Run the `teleop_twist_keyboard` node.
8. Successfully drive and rotate the robot inside Gazebo using your keyboard.

Expected Output Format (Task 4 Submission)

The final output for the combined Task 4 must be submitted as previous tasks. The files must contain:

1. The URDF File: The completed and finalized `.urdf` or `.xacro` file from Task 4-a.
2. Video Demonstration: A screen recording (MP4/MKV format) clearly showing:
 - The bot successfully spawning in the Gazebo environment.
 - The terminal actively running the teleop keys.
 - The bot moving (forward/backward) and turning correctly in response to your key presses.

Implementation and Evaluation

To test your complete implementation, build your workspace, launch your URDF in Gazebo, and run the teleop node in a separate terminal. Check if the physical movement in the simulator matches your expected teleop commands.

IMPORTANT:

- Ensure your left and right wheels are mapped correctly; otherwise, the robot will spin in place or move backward when commanded forward.
- If your bot sinks into the Gazebo floor or flies away upon spawning, return to Task 4-a and fix your `<collision>` and `<inertial>` tags.
- Your evaluation depends on the correct, stable movement of the bot in the video submission.

3. Task 4-c: Plugin documentation

The goal of this task is to research, analyze, and document the core software components that bridge a robot's physical simulation with ROS 2. You will create a comprehensive technical reference guide detailing:

- **5 Sensor Plugins** used in Ignition Gazebo,
- **2 Motion Control Plugins** used in Ignition Gazebo.

Problem Statement

As the robotics division of Cyborg expands its simulation capabilities to more complex autonomous tasks, we require a standardized reference manual for robot actuators and sensors. A URDF model cannot interact with the virtual world or send data to ROS 2 without properly configured plugins.

Your task is to thoroughly document the configuration, parameters, and topic structures of essential Ignition Gazebo plugins. This documentation will serve as a developer's guide for the entire team when integrating sensors (like LiDAR, cameras, or IMUs) and controllers into our simulation pipelines.

Task Requirements

For each of the **7 required plugins** (5 sensors, 2 motion control), your documentation must contain the following five structural components:

1. **Plugin Identity:** The exact syntax for the `<plugin>` tag, including its `filename` and `name` attributes specific to Ignition Fortress.
2. **Functional Description:** A clear, concise explanation of what the plugin does, how it works and when it should be used.
3. **Key Parameters Table:** A breakdown of essential XML tags used inside the plugin configuration (e.g., `<update_rate>`, `<topic>`, `<frame_id>`, `<joint>`) along with their data types and purposes.
4. **Sample XML Snippet:** A clean, properly indented block of URDF/Xacro code demonstrating a standard implementation of the plugin.
5. **ROS 2 Bridge Mapping:** The exact message type (e.g., `geometry_msgs/msg/Twist`, `sensor_msgs/msg/LaserScan`) and the bridge configuration string required to pass this data between Ignition Gazebo and ROS 2.

Expected Output Format (Task 4-c Submission)

You must submit a single, well-formatted **PDF** document named `[YourName_Rollno.]_Plugin_Documentation`.

Note: Everyone is strictly instructed for task 4c to complete it by searching and reading about the plugins in the internet and official documentation. The documentation should be written by you. You can take help of AI for understanding the plugin parameters only.

The layout must follow this exact hierarchy:

Section 1: Motion Control Plugins

1.1 [Plugin Name 1]

1.2 [Plugin Name 2]

Section 2: Sensor Plugins

2.1 [Plugin Name 1]

Deadline: 16 June EOD